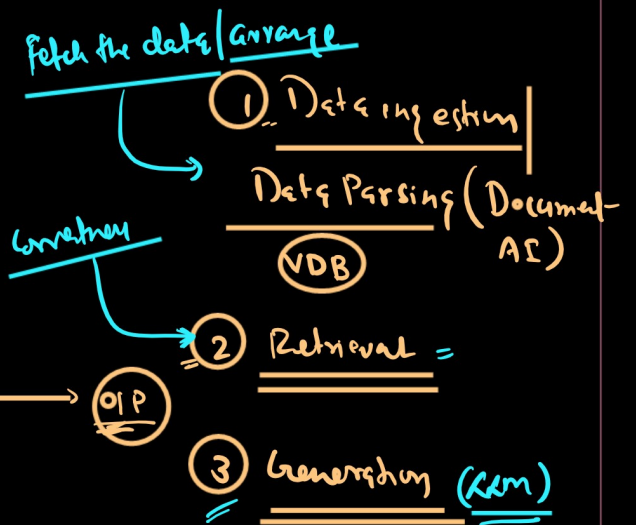
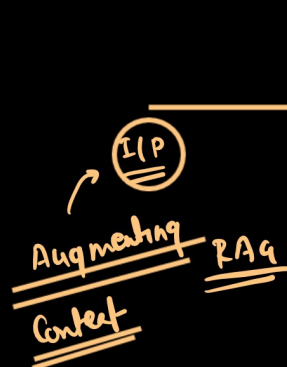
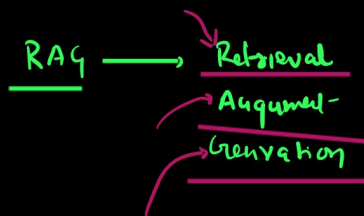


RAG

- ① Encoding & embedding
 - ② transformer / various LLM architecture
 - ③ LLM Access API =
 - ④ Prompting → 8/10 dem
- } 3 months
Fundamental
GenAI
-
- ⑤ RAG (Langchain / Langgraph) 6-8 dem
 - ⑥ Agentic AI (Langchain / Langgraph) 95+



① What is RAG?

② Why RAG?

③ RAG Pipeline → Advance RAG

④ RAG v/s Pretraining

① Data ingestion

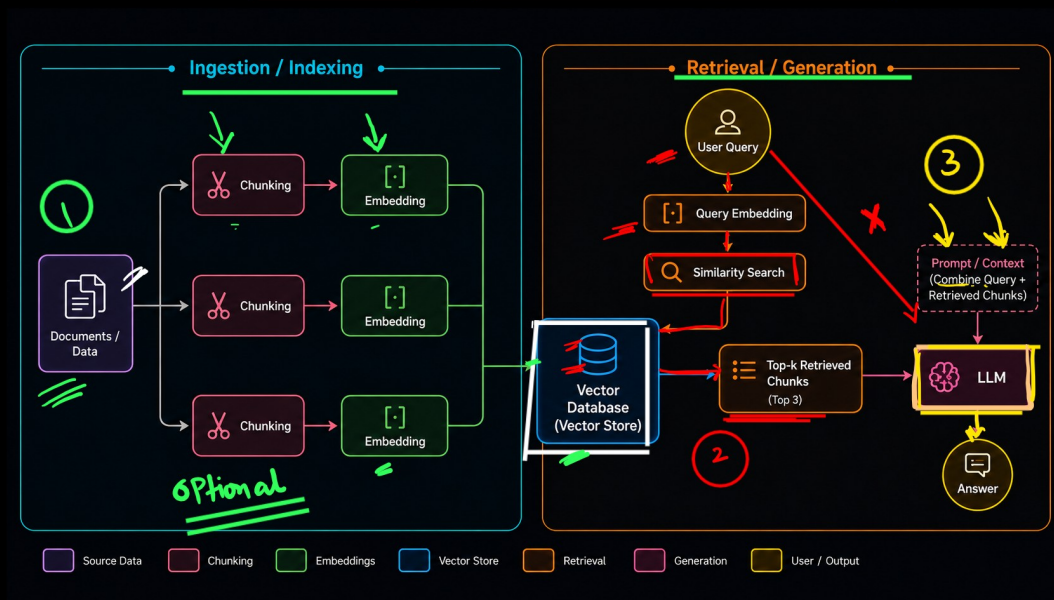
① Data Processing / Document AI

② Vector DB (chunking / embedding)

② Retrieval Pipeline

③ Generation (Prompting)

RAG = Give relevant context to the LLM before answering



RAG is a technique that helps an LLM answer questions using external data such as PDFs, documents, websites, databases, or company knowledge bases.

1. What is RAG?

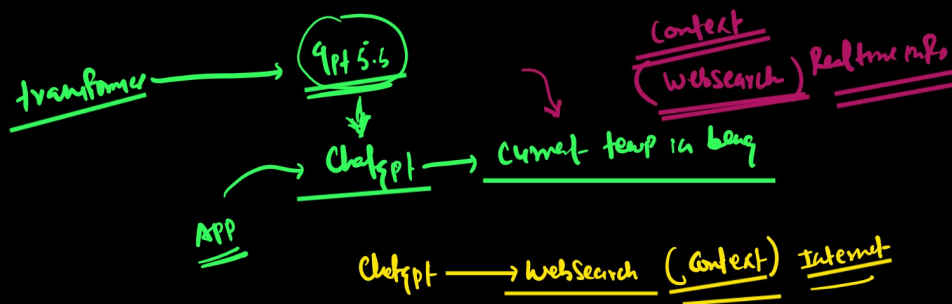
RAG stands for Retrieval-Augmented Generation.

Retrieval = finding relevant information from documents

Augmentation = adding that retrieved information as extra context to the user question

Augmentation = improving the LLM's input by adding retrieved context

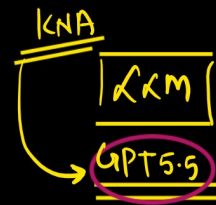
Generation = LLM generates the final answer using that context



Example:

User asks:

"What is the refund policy of our company?"



The LLM may not know this because company policy is private data.

RAG does this:

1. It searches the company policy document.
2. It retrieves the relevant refund policy section.
3. It adds that retrieved section as extra context to the user question.
4. It gives the context + question to the LLM.
5. The LLM generates the final answer based on that context.

For example:

User question:

"What is the refund policy?"

Retrieved context:

"Refunds are allowed within 7 days of purchase."

Augmented prompt:

Question: What is the refund policy?

Context: Refunds are allowed within 7 days of purchase.

Final answer:

The refund policy allows refunds within 7 days of purchase.

(Similarity search) mathematical operation

Augmentation

LLM

LLMs do not have access to the latest/private data

We retrieve relevant context from documents

We give the LLM the context + question

The LLM gives a grounded answer

2. Why RAG?

We use RAG because normal LLMs have some limitations.

Problem 1: LLMs do not have private data

Problem 2: LLMs may not have latest data

Problem 3: LLMs can hallucinate

Sometimes LLMs generate confident but wrong answers.

RAG reduces hallucination because the answer is based on retrieved context.

Example:

User asks: "What is the refund policy of ABC Company?"

Without RAG, the LLM may confidently say:

"ABC Company offers a 30-day refund policy."

But in the actual company document, the policy says: "Refunds are allowed only within 7 days."

So the LLM gave a confident answer, but it was wrong.

This is called hallucination.

Problem 4: Fine-tuning is not always the right solution

For every new document, we cannot fine-tune the model again and again.

RAG is better when data changes frequently.

Example:

If HR policy changes every month,
you should update documents in RAG,
not fine-tune the model every month.

Chatbot - (GPTs)

XYZ Company

Pharma (Data)

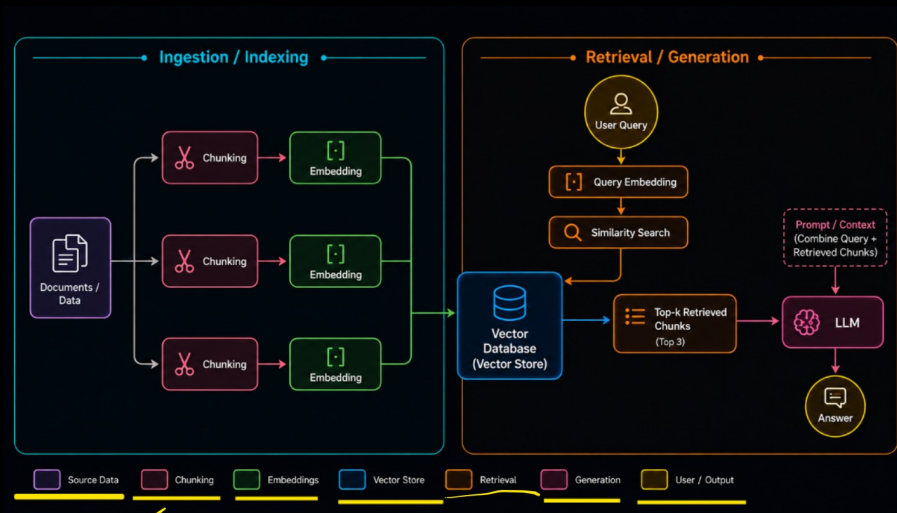
Private / Real /
Proprietary data

(GPTs / Claude /
Gemini)

RAG

3. Pipeline to Implement RAG

RAG helps an LLM answer questions using external documents. First, we parse the documents and extract clean text. Then we split the text into chunks, convert chunks into embeddings, and store them in a vector database. When the user asks a question, we retrieve the most relevant chunks and give them to the LLM along with the question. This



RAW

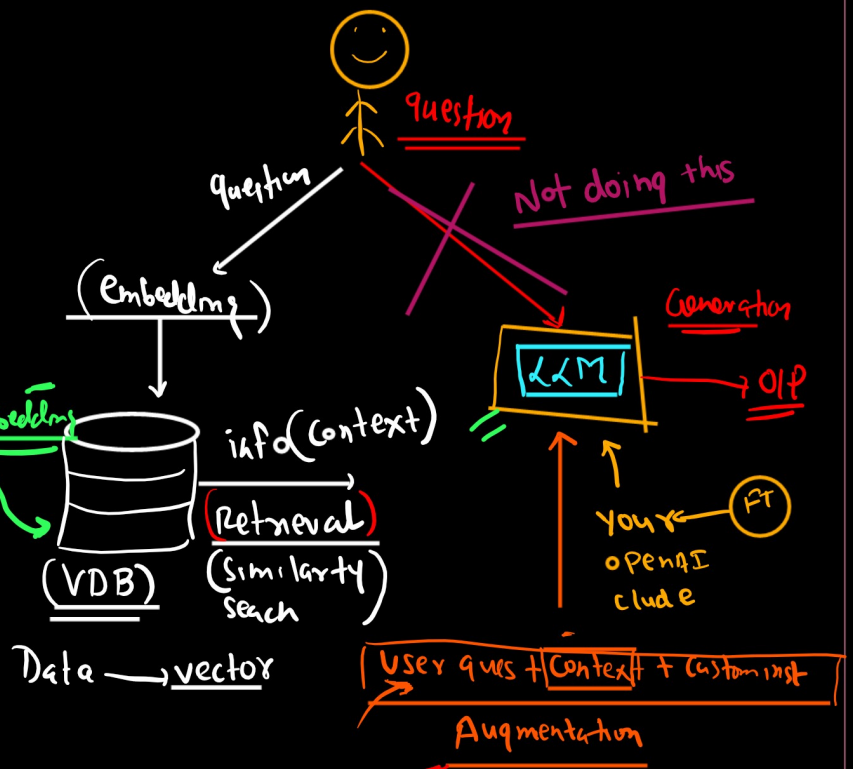
Data Source =

- ① PDF
- ② docx
- ③ internal DB
- ④ DB
- ⑤ web
- ⑥ internal portal
- ⑦ cloud architecture
- ⑧ audio/image/video

Data Parsing =

Chunking
Data into small pieces

Data ingestion



RAG vs Fine-tuning

Quick Difference

 Column	 RAG	 Fine-tuning
 Main purpose	Gives the model external knowledge/context	Teaches the model new behavior, style, or format
 Model weights change?	No	Yes
 Cost	Usually lower	Usually higher
 Data update	Easy — update documents/vector DB	Harder — retrain or fine-tune again

The diagram is divided into three main sections, each with a colored header and a corresponding icon:

- 1. Nature / Modality (Green Header):** This section lists four types of data with their respective icons: Text (speech bubble), Image (landscape photo), Audio (speaker), and Video (clapperboard).
- 2. Storage Format (Blue Header):** This section lists the file formats for each data type: PDF (document icon), DOCX (Word document icon), CSV (spreadsheet icon), JSON (code icon), YAML (document icon), MP3 (music note icon), and MP4 (video player icon).
- 3. Structure (Purple Header):** This section categorizes data into three types based on structure:
 - Structured data (Green Box):** Includes Tables, CSV, and SQL.
 - Semi-structured data (Yellow Box):** Includes JSON, YAML, and HTML.
 - Unstructured data (Red Box):** Includes PDFs, Images, Audio, Video, and Reports.

Handwritten notes in red ink are present: "Graphical" with an arrow pointing to the "Image" icon, and "Graphical" with an arrow pointing to the "Structure" header.

- Llama Parser
- Docling
- Google Document AI
- Amazon Textract
- python-docx
- PyMuPDF
- pdfplumber
- Parsio
- Unstructured.io
- Azure Document Intelligence

PAD
Frame | library extracting the data

PAID

web → html

<table>
<text>

TEXT modality

my name is sunny.

Image Modality

= Data Format / File Format -

PDF

DOCX

Append

TABLE

standing

Filebuds

Average

FCN

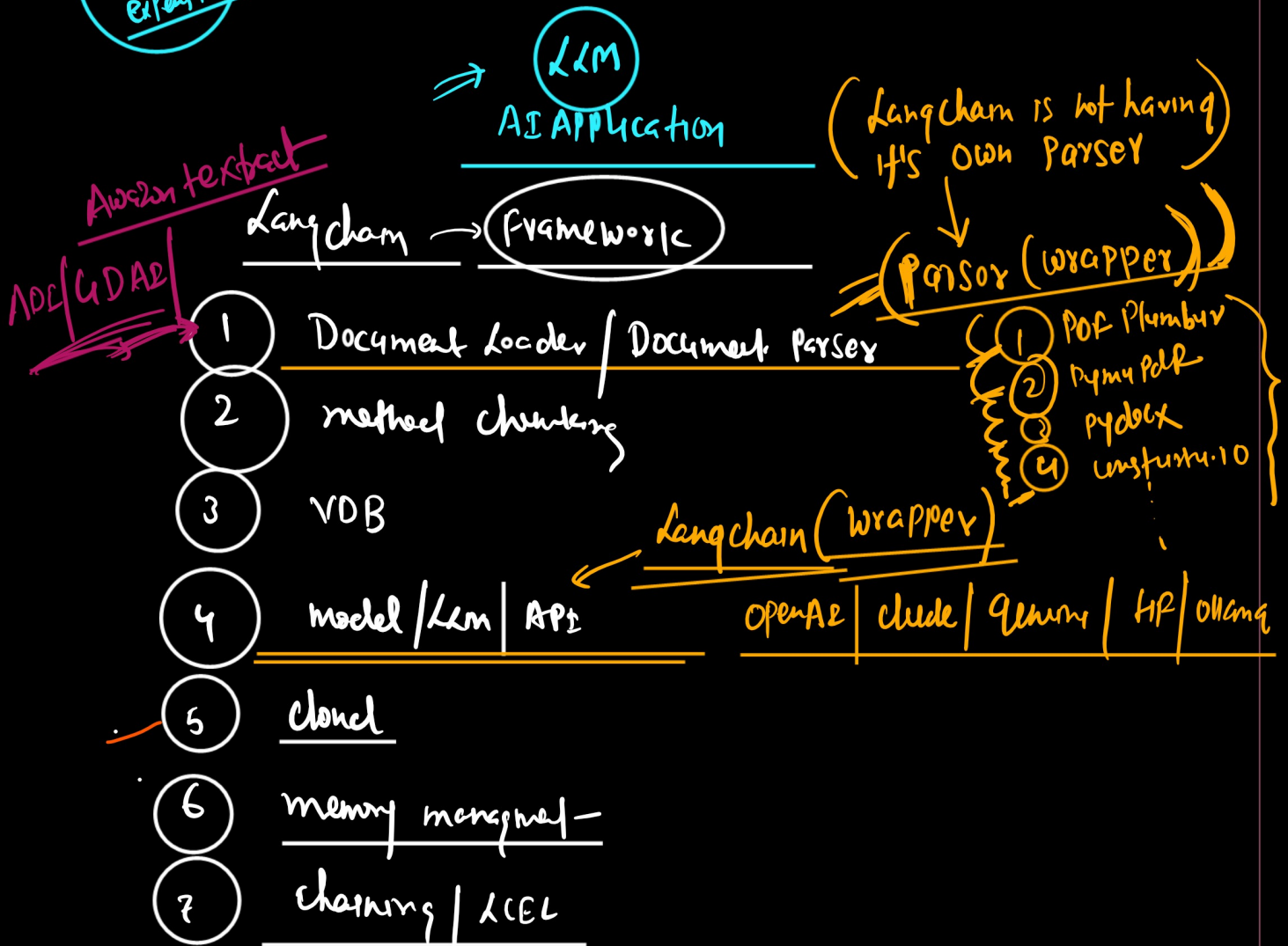
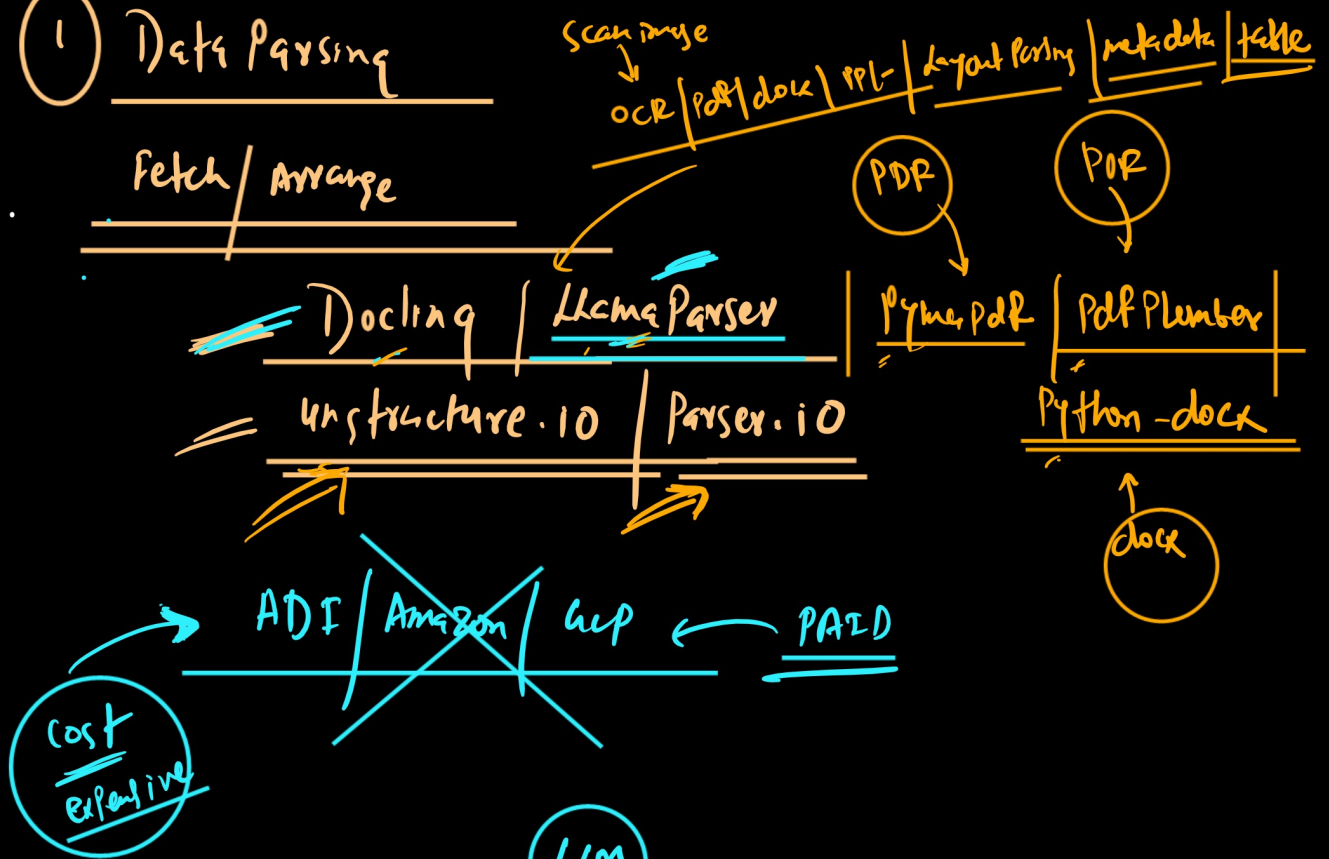


• x/x

Data Parsing

TABULAI2

1 Data Parsing



LangChain does provide document loaders, but in most cases LangChain is not doing the low-level parsing by itself. It mainly provides a common wrapper/interface over different parsing libraries and services.

LangChain itself is not primarily a parsing library.

LangChain provides document loaders, but many of those loaders internally use existing parsing libraries such as PyPDF, PyMuPDF, pdfplumber, python-docx, Unstructured, and others.

So LangChain acts like a wrapper or integration layer that gives us a common Document format: `page_content` + `metadata`.

This makes it easy to plug parsed documents into the RAG pipeline.

Vector Databases & Search (RAG Backbone)

- FAISS
- Chroma
- Qdrant
- Supabase
- pgvector
- Milvus
- Pinecone
- AWS OpenSearch
- Azure AI Search

LangChain wraps different parsing libraries so developers can use many data sources through one common interface. Its goal is not only parsing, but connecting parsing with chunking, embeddings, vector databases, retrievers, prompts, and LLMs. This makes the full RAG pipeline easier to build and maintain.

•

Fetch → img | text | net | layout
→ mdl | .json

google cloud / GPU ← ~~training~~ PAGE
local / CPU